

Smoke100 Eval - Non-Determinism Analysis

Source runs:

Run	MLflow run id
run1	30bca13482944fc0a0ca4eed34a51390
run2	2bacd8bb488a4e79986749e1703098fa
run3	5abe9d81bba14d91af922a1ca0a52f4b

Runtime-Equivalence Rules Applied

These rules were checked against the `analyze_transactions` signature and docstring in `ai-orchestrator/packages/mcp/src/ai_mcp/tools.py`. They are used only for deterministic `ToolParameterScorer` scoring. Raw `expected_tool_calls` and `actual_tool_calls` stay unchanged for review/reporting.

The point is simple: if two tool calls behave the same at runtime, the scorer should not count them as different.

Rule	Why applied?
Apply these equivalence rules only to <code>analyze_transactions</code> .	This is the only tool where we verified these defaults and aliases from the ai-orchestrator source.
Treat <code>sort=MAIN_DATE</code> , <code>MAIN_DATE_ASC</code> , and <code>MAIN_DATE_DESC</code> as the same as <code>EXECUTION_DATE</code> , <code>EXECUTION_DATE_ASC</code> , and <code>EXECUTION_DATE_DESC</code> .	<code>sort</code> orders the returned transaction rows. The ai-orchestrator says <code>MAIN_DATE</code> sorting is interpreted as execution-date sorting, so those names behave the same.
Ignore <code>sort_by</code> when <code>group_by</code> is missing or <code>group_none</code> .	<code>sort_by</code> is different from <code>sort</code> : it orders aggregation groups, not individual transaction rows. If the call is not grouping transactions, there are no groups to sort, so <code>sort_by</code> has no effect.
If the test expected <code>group_by=group_none</code> and the agent omitted <code>group_by</code> , count it as equivalent.	<code>group_none</code> is the backend default. Omitting it and writing it explicitly lead to the same call behavior.
If the test expected <code>sort_by=total_sum_desc</code> for a grouped transaction analysis and the agent omitted <code>sort_by</code> , count it as equivalent.	<code>total_sum_desc</code> is the backend default for sorting aggregation groups. This does not replace <code>sort</code> for questions about latest/largest/smallest individual transactions. If there is no grouping, <code>sort_by</code> is ignored instead.
If the test expected <code>visualization_type=SUMMARY</code> and the agent omitted <code>visualization_type</code> , count it as equivalent.	<code>SUMMARY</code> is the backend default. The agent does not need to say it explicitly for the call to behave that way.
If the test expected <code>exclude_own_transfers=True</code> and the agent omitted <code>exclude_own_transfers</code> , count it as equivalent.	The backend excludes own transfers by default. Omitting the flag still means <code>True</code> .
If the test expected <code>size=1000</code> and the agent omitted <code>size</code> , count it as equivalent.	The tool docstring says the upstream transaction fetch defaults to <code>1000</code> . This is different from <code>limit</code> , which controls how many rows are returned after filtering/sorting.
If the test expected <code>products_filter_mode</code> and the agent omitted it, fill the value the backend will use: <code>PRODUCT_SELECTION</code> when <code>account_ids</code> is present, otherwise <code>ALL</code> .	The backend chooses this mode based on whether account/card IDs were passed. This only scores as equivalent when the expected value is the same as that backend-chosen value. If the test expects a different mode, omission is still a real mismatch.
If <code>transaction_collection_id</code> is present in the <code>analyze_transactions</code> input arguments, ignore filter parameters on that same call: dates, direction, categories, search text, transaction types, amount filters, account IDs, product filter mode, <code>exclude_own_transfers</code> , <code>size</code> , <code>limit</code> , and <code>sort</code> . This does not refer to the <code>transaction_collection_id</code> field returned in the tool output. We did not observe this input pattern in the three current reruns.	The ai-orchestrator skips the upstream fetch only when the agent passes a previous collection ID back into <code>analyze_transactions</code> . In that re-aggregation path, filters are ignored; only <code>group_by</code> , <code>sort_by</code> , and <code>visualization_type</code> still matter. A normal first fetch also returns a <code>transaction_collection_id</code> , but that output field does not mean the call skipped fetching.

Summary Of Findings

After re-scoring the three runs with runtime-equivalence enabled:

Metric	Result
<code>agent_routing_score</code> changed across runs	0/100 cases
<code>tool_usage_score</code> changed across runs	2/100 cases
<code>tool_parameter_score</code> changed across runs	14/100 cases
Tool-parameter-scored rows with raw parameter variation	40/64 rows
Tool-parameter-scored rows with relaxed parameter variation	36/64 rows
Apparent drift removed by relaxation	4 cases

Relaxation removed false-positive drift for these cases:

- `smoke-017`
- `smoke-023`
- `smoke-056`
- `smoke-088`

Expected defaults still present in the test set:

Test case	Expected default	Action
<code>smoke-014</code>	<code>visualization_type='SUMMARY'</code>	Remove from <code>expected_tool_calls</code> unless this test intentionally checks default emission.
<code>smoke-023</code>	<code>sort_by='total_sum_desc'</code>	Remove from <code>expected_tool_calls</code> unless this test intentionally checks default emission.

The remaining issue is not routing. It is mostly tool-argument generation: the model inconsistently emits filters that the prompt/tool descriptions leave under-specified.

Actionable Items fro Review

1. Card lookup filters

Test cases:

- `smoke-006`: debit card limits; `getCards.type=DEBIT` omitted in run1. Still gets the right cardID for the second tool cal since it is only one card.
- `smoke-008`: debit card expiry; `getCards.type=DEBIT` omitted in run1/run2.
- `smoke-009`: debit card blocked status; `getCards.type=DEBIT` omitted in run1.
- `smoke-050`: virtual active card; `isVirtual=True` and/or `state=ACTIVE` omitted depending on run.
- `smoke-096`: cards assigned to savings account; `getCards.accountId` omitted in run3.
- `smoke-097`: credit card spend limit; `getCards.type=CREDIT` omitted in run2.
- `smoke-098`: credit card temporary limits; `getCards.type=CREDIT` omitted in run1/run2.

Prompt cross-check:

- Current `getCards` prompt says: pass card-type filter when the user names a card kind.
- It does not explicitly say to pass `state=ACTIVE` for current-card questions.
- It does not explicitly say to pass `accountId` after `getAccounts` when the user asks for cards linked to a specific account.
- It says `virtual -> virtual`, but the tool parameter is `isVirtual`; make that mapping explicit.

2. Account ownership/type filters

Test cases:

- `smoke-060`: savings account existence; `getAccounts.type=SAVING` and `ownedByCurrentUser=True` omitted in run1/run2.
- `smoke-025`: savings account interest; account filter was correct, but the expected mixed `knowledge_search` call was not always made. See item 6.

Prompt cross-check:

- Current `getAccounts` prompt says to use the tool for balances, IBAN, account number, and ownership.
- It does not mention `ownedByCurrentUser` as a parameter.
- Tool description has an ownership example, but the agent does not consistently apply it to "my savings account".

3. Transaction category/subcategory mappings for common intents

Test cases:

- `smoke-042`: fuel spend; `sub_category=[ 'GAS_FUEL' ]` omitted in run2.
- `smoke-054`: ATM/cash withdrawal; runs alternated between `types=[ 'ATM' ]`, `main_category=[ 'WITHDRAWAL' ]`, broader `sub_category`, and wrong `visualization_type`.
- `smoke-043`: average monthly food spend; `direction=OUTGOING` or `group_by=group_by_month` drifted.

Prompt cross-check:

- Current `analyze_transactions` prompt says direction is mandatory for one-sided intent.
- It says category breakdowns should group by category and time-series should group by month.
- It does not provide concrete mappings for common business-language intents like fuel, ATM withdrawal, cash withdrawal, or average monthly spend.

4. Superlative sorting rules for transaction details

Test case:

- `smoke-055`: largest single expense last month; run1/run3 used `sort=EXECUTION_DATE_DESC` and `limit=100` instead of `sort=AMOUNT_DESC` and `limit=1`.

Prompt cross-check:

- Current prompt says to match `limit` to the ask.
- It does not explicitly map "largest/highest/biggest single expense" to amount descending sort.

5. Period comparisons

Test case:

- `smoke-066`: compare December 2025 and January 2026 spending; run2 only called `analyze_transactions` for

December, so tool usage and tool parameters both dropped.

Prompt cross-check:

- Current prompt says `analyze_transactions` covers comparisons.
- It does not explicitly instruct the agent to call the tool once per named comparison period.

6. Mixed product plus KB expectations

Test case:

- `smoke-025`: current savings-account interest rate; expected calls include `getAccounts` and `knowledge_search`.
run1/run2 only called `getAccounts`; run3 also called `knowledge_search`.

Prompt cross-check:

- Current `getAccounts` prompt says: if the user mentions a specific interest rate, present the rate from tool data without commentary.
- It does not say when to use `knowledge_search` for current savings interest-rate policy/guidance.
- The deterministic parameter scorer still scores the `knowledge_search.question` key in this mixed API case, even though `knowledge_search` parameters are fuzzy and excluded only for KB-only rows.

7. Test set issues

Test cases:

- `smoke-014`: remove `visualization_type='SUMMARY'` from expected `analyze_transactions` parameters.
- `smoke-023`: remove `sort_by='total_sum_desc'` from expected `analyze_transactions` parameters.

Reason:

- These are source-backed runtime defaults. Keeping them in `expected_tool_calls` inflates the scorer denominator and makes harmless omission look important.